

MARAN PMS — Prompt 55: Motor Financiero Común (Roadmap Post-Producción)

Cuándo implementar

Después de que el sistema esté funcionando en producción. Este prompt es el diseño del motor financiero unificado descrito en el documento "Maran Motor Financiero Común". No tocar hasta tener al menos 30 días de operación real con datos reales.

Por qué esperar

- Las decisiones de diseño se validan con datos reales de operación
 - El riesgo de romper módulos en producción es alto
 - El sistema actual funciona — este prompt lo hace mejor, no lo arregla
-

DIAGNÓSTICO DEL ESTADO ACTUAL

Lo que existe y funciona bien (NO tocar)

- `cash_movements` + `registerCashMovement()` — registro operativo de caja, todos los módulos lo usan
- `cashShifts` + `closeShift()` — gestión de turnos con arqueo
- `cashClosingSummaries` — resumen por método al cerrar turno
- `payments` — pagos de reservas hoteleras
- `spa_payments` — pagos del spa
- `event_payments` — pagos de eventos
- `charges` — cargos de reservas hoteleras

El problema central: dos mundos paralelos

Mundo 1: `cash_movements` (registro operativo)
← todos los módulos escriben aquí al cobrar 

Mundo 2: `payments/spa_payments/event_payments` (registro contable)
← cada módulo tiene su propia tabla 

Problema: no hay hilo conductor entre ambos mundos.
Un pago en `spa_payments` no sabe a qué `cash_movement` corresponde.

Lo que falta para el motor financiero real

1. Tabla `folios` — cuenta viva por entidad (reserva, mesa, spa, grupo, evento, empresa)
2. Tabla `folio_movements` — todo evento financiero unificado
3. Vínculo entre `folio_movements` y `cash_movements` (un pago genera ambos)
4. Conciliación formal
5. Factura naciendo desde el folio

FASE 1 — TABLA DE FOLIOS

Nueva tabla `folios`

```
CREATE TABLE folios (  
  id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),  
  codigo TEXT NOT NULL UNIQUE,           -- FOL-2026-00001 (correlativo)  
  
  -- Entidad propietaria del folio (solo una activa)  
  entity_type TEXT NOT NULL,              -- 'reservation' | 'restaurant_order' |  
                                          -- 'spa_account' | 'group' | 'event' |  
                                          -- 'company' | 'agency'  
  entity_id VARCHAR NOT NULL,             -- ID de la entidad  
  
  -- Estado  
  status TEXT NOT NULL DEFAULT 'open',    -- 'open' | 'closed' | 'invoiced'  
  
  -- Saldos calculados (desnormalizados para performance)  
  total_charges DECIMAL(12,2) DEFAULT 0,  
  total_payments DECIMAL(12,2) DEFAULT 0,  
  balance DECIMAL(12,2) DEFAULT 0,        -- total_charges - total_payments
```

```

-- Metadatos
opened_at TIMESTAMP DEFAULT NOW(),
closed_at TIMESTAMP,
closed_by TEXT,
notes TEXT,
created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_folios_entity ON folios(entity_type, entity_id);
CREATE INDEX idx_folios_status ON folios(status);

```

En `shared/schema.ts` :

```

export type FolioEntityType =
  "reservation" | "restaurant_order" | "spa_account" |
  "group" | "event" | "company" | "agency";

export type FolioStatus = "open" | "closed" | "invoiced";

export const folios = pgTable("folios", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
  codigo: text("codigo").notNull().unique(),
  entityType: text("entity_type").$type<FolioEntityType>().notNull(),
  entityId: varchar("entity_id").notNull(),
  status: text("status").$type<FolioStatus>().notNull().default("open"),
  totalCharges: decimal("total_charges", { precision: 12, scale: 2 }).default("0"),
  totalPayments: decimal("total_payments", { precision: 12, scale: 2 }).default("0"),
  balance: decimal("balance", { precision: 12, scale: 2 }).default("0"),
  openedAt: timestamp("opened_at").defaultNow(),
  closedAt: timestamp("closed_at"),
  closedBy: text("closed_by"),
  notes: text("notes"),
  createdAt: timestamp("created_at").defaultNow(),
});

```

FASE 2 — TABLA DE MOVIMIENTOS DE FOLIO

```

CREATE TABLE folio_movements (
  id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),
  folio_id VARCHAR NOT NULL REFERENCES folios(id),

  -- Tipo de movimiento
  type TEXT NOT NULL,

```

```

-- 'charge'           → cargo al folio (alojamiento, consumo, servicio)
-- 'payment'          → pago que cancela saldo
-- 'advance'          → anticipo (pago antes de la prestación)
-- 'discount'         → descuento aplicado
-- 'adjustment'       → ajuste manual con motivo
-- 'transfer_in'      → recibe monto desde otro folio
-- 'transfer_out'     → envía monto a otro folio
-- 'void'             → anulación de movimiento anterior

amount DECIMAL(12,2) NOT NULL,          -- siempre positivo
description TEXT NOT NULL,

-- Referencia al origen
source_type TEXT,          -- 'reservation_charge' | 'restaurant_order' | 'spa_servi
source_id VARCHAR,        -- ID del registro origen

-- Para pagos
payment_method TEXT,      -- método de pago si es type='payment'
cash_movement_id VARCHAR, -- vínculo con cash_movements (si generó movimiento

-- Para transferencias
related_folio_id VARCHAR, -- folio origen/destino en transferencias

-- Para anulaciones
voided_movement_id VARCHAR, -- movimiento que anula
void_reason TEXT,

-- Auditoría
registered_by TEXT,
receipt_type TEXT,      -- tipo de comprobante si aplica
created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_folio_movements_folio ON folio_movements(folio_id);
CREATE INDEX idx_folio_movements_type ON folio_movements(type);

```

FASE 3 — ESTRATEGIA DE MIGRACIÓN (sin romper lo existente)

Principio: construir encima, no reemplazar

Las tablas existentes (`payments` , `spa_payments` , `event_payments` , `charges`) **NO se eliminan**. Se crea la capa de folios encima.

Paso 1: Crear tablas folios y folio_movements

Paso 2: Crear función getOrCreateFolio(entityType, entityId) **en storage:**

```
async getOrCreateFolio(entityType: FolioEntityType, entityId: string): Promise<Fo
  const [existing] = await db.select().from(folios)
    .where(and(
      eq(folios.entityType, entityType),
      eq(folios.entityId, entityId),
      eq(folios.status, "open")
    ));

  if (existing) return existing;

  const codigo = await generateFolioCodigo();
  const [created] = await db.insert(folios).values({
    id: randomUUID(),
    codigo,
    entityType,
    entityId,
    status: "open",
  }).returning();

  return created;
}
```

Paso 3: Modificar registerCashMovement **para que OPCIONALMENTE** también genere un folio_movement :

```
async registerCashMovement(
  area, sourceType, sourceId, sourceLabel, paymentMethod, amount, movementType,
  registeredBy?, receiptType?,
  folioOptions?: { entityType: FolioEntityType; entityId: string } // ← nuevo pa
): Promise<CashMovement> {
  const turno = await this.getOrCreateActiveTurno(area);
  const [movement] = await db.insert(cashMovements).values({ ... }).returning();

  // Si viene folioOptions, también registrar en folio_movements
  if (folioOptions && movementType === "income") {
    const folio = await this.getOrCreateFolio(folioOptions.entityType, folioOptio
    await db.insert(folioMovements).values({
      folioId: folio.id,
      type: "payment",
      amount,
      description: sourceLabel,
      sourceType,
```

```

        sourceId,
        paymentMethod,
        cashMovementId: movement.id,
        registeredBy,
        receiptType,
    });
    // Actualizar balance del folio
    await this.recalcFolioBalance(folio.id);
}

return movement;
}

```

Paso 4: Migrar módulo por módulo:

1. Hotel (reservas) — el más importante, empezar aquí
2. Restaurant (órdenes)
3. Spa (cuentas)
4. Grupos (folio maestro)
5. Eventos
6. Empresas/Agencias

Para cada módulo: al crear un cargo o registrar un pago, también llamar a
`getOrCreateFolio + crear folio_movement .`

FASE 4 — VISTAS Y REPORTES DESDE FOLIOS

Una vez que todos los módulos generan `folio_movements` , los reportes dejan de hacer JOINS complejos entre múltiples tablas y simplemente consultan `folios + folio_movements .`

```

-- Saldo de una reserva
SELECT balance FROM folios
WHERE entity_type = 'reservation' AND entity_id = $reservationId;

-- Todos los folios con saldo pendiente de empresas
SELECT f.*, e.razon_social
FROM folios f
JOIN companies e ON f.entity_id = e.id
WHERE f.entity_type = 'company' AND f.balance > 0.01;

```

```
-- Movimientos del día por tipo
SELECT type, SUM(amount)
FROM folio_movements
WHERE DATE(created_at) = $fecha
GROUP BY type;
```

FASE 5 — CONCILIACIÓN

Nueva tabla reconciliations para el día a día:

```
CREATE TABLE reconciliations (
  id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),
  fecha DATE NOT NULL,
  area TEXT NOT NULL,
  shift_id VARCHAR REFERENCES cash_shifts(id),

  -- Lo que dice el sistema
  sistema_efectivo DECIMAL(12,2),
  sistema_debito DECIMAL(12,2),
  sistema_credito DECIMAL(12,2),
  sistema_transferencia DECIMAL(12,2),
  sistema_mercadopago DECIMAL(12,2),

  -- Lo que se verificó físicamente
  verificado_efectivo DECIMAL(12,2),
  -- (tarjetas y transferencias se verifican contra el resumen del POS/banco)

  -- Diferencias
  diferencia_efectivo DECIMAL(12,2) GENERATED ALWAYS AS
    (verificado_efectivo - sistema_efectivo) STORED,

  -- Estado
  status TEXT DEFAULT 'pending', -- 'pending' | 'reconciled' | 'discrepancy'
  notes TEXT,
  reconciled_by TEXT,
  reconciled_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

CUENTA CORRIENTE DE EMPRESAS Y AGENCIAS

La tabla `account_movements` ya existe en el sistema. El motor financiero la conecta con los folios:

```
// Al cerrar un folio de empresa con saldo pendiente:
// 1. Folio queda en status 'closed' con balance > 0
// 2. Se crea account_movement de tipo 'cargo' en la cuenta corriente de la empre
// 3. Cuando la empresa paga, se crea account_movement de tipo 'pago'
// y se crea folio_movement de tipo 'payment' para cerrar el folio
```

ORDEN DE IMPLEMENTACIÓN RECOMENDADO

Mes 1 (post-producción):

- ✓ Validar con datos reales que el flujo de caja actual es correcto

Mes 2:

- Crear tablas folios + folio_movements
- Implementar `getOrCreateFolio` y `recalcFolioBalance`
- Migrar módulo hotel (reservas)
- Verificar que balance de folios coincide con `charges - payments` existentes

Mes 3:

- Migrar restaurant, spa, grupos, eventos
- Vista de "Folio" unificada en cada módulo

Mes 4:

- Conciliación formal
- Reportes desde folios
- Cuenta corriente conectada con folios

Mes 5+:

- Deprecar queries directas a `payments/spa_payments/event_payments`
 - Todo desde `folio_movements`
-

NOTAS IMPORTANTES

- **No eliminar** `payments`, `spa_payments`, `event_payments` hasta que la migración esté 100% verificada con datos reales
- El campo `balance` en `folios` es desnormalizado para performance — actualizarlo en cada `insert/update` de `folio_movements` con `recalcFolioBalance()`

- Los códigos de folio (FOL-2026-00001) son correlativos por año, igual que los de reserva
 - La factura electrónica (ARCA/AFIP) se conecta en esta fase — nace desde el folio, no desde el pago
 - Este motor habilita escalar el sistema a múltiples hoteles porque el folio es agnóstico del hotel
-

ENTIDADES FINALES DEL MOTOR

folios	← cuenta viva por entidad
folio_movements	← todo evento financiero
cash_movements	← registro operativo de caja (existente, se mantiene)
cash_shifts	← turnos de caja (existente, se mantiene)
cash_closing_summaries	← resúmenes de cierre (existente, se mantiene)
reconciliations	← conciliación formal (nueva)
account_movements	← cuenta corriente empresas/agencias (existente, se man
sales_invoices	← facturas (existente, conectar con folios)